BCS 변환

개요

BCS(Ben-Sasson, Chiesa, Spooner) 변환은 Interactive Oracle Proof(IOP)를 비상호작용 zkSNARK로 변환하는 핵심 기법입니다. 이 변환을 통해 상호작용이 필요한 IOP를 한 번의 증명 생성으로 여러 검증자가 독립적으로 검증할 수 있는 시스템으로 만들 수 있습니다.

BCS 변환의 원리

기본 아이디어

- **Fiat-Shamir 변환의 확장**: 공개 코인 IOP에 적용
- **해시 함수 활용**: 무작위성을 해시 함수로 대체
- **머클 트리 구조**: 오라클 쿼리를 효율적으로 처리

변환 과정

1단계: 오라클 커밋

```
각 라운드 i에서:
- 증명자가 오라클 πᵢ 생성
- 머클 트리 Tᵢ = MerkleTree(πᵢ) 구성
- 루트 해시 rᵢ = Root(Tᵢ) 계산
```

2단계: 챌린지 생성

```
각 라운드 i에서:

- 이전 상태 state_{i-1} 과 루트 해시 r_i를 입력으로 사용

- 챌린지 c_i = Hash(state_{i-1} || r_i) 생성

- 상태 업데이트 state_i = Hash(state_{i-1} || r_i || c_i)
```

3단계: 쿼리 응답

```
최종 라운드에서:

- 모든 쿼리 위치 Q = {q_1 , q_2 , ..., q_m} 결정

- 각 쿼리에 대한 응답과 머클 증명 생성

- 증명 π = {(r_1 , ..., r_k), {(π_i[q_j], path_{i,j})}_{i,j}}
```

머클 트리와 해시 체인

머클 트리의 역할

- **데이터 무결성**: 오라클 내용의 변조 방지
- **효율적 쿼리**: O(log n) 크기의 증명으로 특정 위치 값 검증
- **배치 처리**: 여러 쿼리를 효율적으로 처리

해시 체인 구조

```
state_0 = Hash(공개 입력)

state_1 = Hash(state_0 || r_1 || c_1 )

state_2 = Hash(state_1 || r_2 || c_2 )
```

```
...

state_k = Hash(state_{k-1} || r_k || c_k)
```

보안성 보장

- **바인딩**: 머클 트리 루트로 오라클 내용 고정
- **무작위성**: 해시 함수의 출력으로 예측 불가능한 챌린지 생성
- **일관성**: 해시 체인으로 전체 프로토콜의 일관성 보장

영지식성 보존

시뮬레이터 구성

1. **무작위 오라클 모델**: 해시 함수를 무작위 오라클로 모델링
2. **프로그래밍 가능성**: 시뮬레이터가 해시 함수 출력을 제어
3. **지연 샘플링**: 필요한 시점에만 무작위 값 생성

영지식성 증명 스케치

```
시뮬레이터 S:

1. 공개 입력 x만으로 시작

2. 각 라운드에서 무작위 루트 해시 r_i 생성

3. 해시 함수를 프로그래밍하여 적절한 챌린지 생성

4. 쿼리 시점에 일관된 응답 생성

5. 실제 증명과 구별 불가능한 시뮬레이션 증명 출력
```

작업 증명(PoW) 최적화

문제점

- **해시 계산 비용**: 많은 해시 연산으로 인한 성능 저하
- **메모리 사용량**: 큰 머클 트리로 인한 메모리 부담

최적화 기법

1. 배치 해싱

```
// 여러 값을 한 번에 해싱
vector<hash_t> batch_hash(const vector<field_t>&  values) {
// SIMD 명령어를 활용한 병렬 해싱
return  parallel_hash(values);
}
```

2. 지연 머클 트리 구성

```
class LazyMerkleTree {
// 필요한 경우에만 노드 계산
hash_t get_root() {
if (!root_computed) {
compute_root_lazy();
}
return root;
```

```
}

};
```

3. 캐시 최적화

- **지역성 활용**: 메모리 접근 패턴 최적화
- **프리페칭**: 예상되는 데이터 미리 로드
- **압축**: 중간 결과의 효율적 저장

구현 세부사항

해시 함수 선택

- **Blake2b**: 빠른 속도와 높은 보안성
- **SHA-3**: 표준화된 보안성
- **Poseidon**: zk-friendly 해시 함수 (특수 용도)

머클 트리 최적화

```
template<typename FieldT>

class OptimizedMerkleTree {

private:

vector<FieldT> leaves;

vector<hash_t> internal_nodes;

size_t height;

public:
```

```
// 병렬 구성
void build_parallel() {
#pragma omp parallel for
for (size_t i = 0; i < leaves.size(); ++i) {
// 리프 노드 해싱
}
// 상향식 병렬 구성
for (int level = height - 1; level >= 0; --level) {
#pragma omp parallel for
for (size_t i = 0; i < (1 << level); ++i) {
internal_nodes[level][i] =
hash(internal_nodes[level+1][2*i],
internal_nodes[level+1][2*i+1]);
}
}
}
// 효율적 증명 생성
MerkleProof generate_proof(size_t index) {
MerkleProof proof;
size_t current_index = index;
for (int level = height; level > 0; --level) {
size_t sibling_index = current_index ^ 1;
proof.siblings.push_back(
```

```
internal_nodes[level][sibling_index]

);

current_index >>= 1;

}

return proof;

}

};
```

보안성 분석

사운드니스

- **해시 함수 보안성**: 충돌 저항성과 일방향성에 의존
- **머클 트리 바인딩**: 루트 해시로 전체 오라클 내용 고정
- **Fiat-Shamir 보안성**: 무작위 오라클 모델에서의 보안성

영지식성

- **시뮬레이터 존재**: 무작위 오라클 모델에서 완벽한 시뮬레이션
- **정보 누출 방지**: 쿼리 응답에서 추가 정보 누출 없음

실용적 고려사항

- **해시 함수 선택**: 양자 후 보안성 고려
- **매개변수 설정**: 보안 수준과 효율성의 균형
- **구현 보안**: 사이드 채널 공격 방지

다음 단계

이제 기초 이론을 마쳤으므로, 수학적 기초 섹션에서 zkSNARK 구현에 필요한 수학적 배경을 학습하겠습니다.

← 이전: IOP와 PCP 관계 | 다음: 수학적 기초 →